

ALGORITHM:

Step 1: Start.

Step 2: Declare choice as an integer.

Step 3: Display the menu

Step 4: Read the user's choice.

Step 5: Use a switch statement:

- If choice is 1, call decreasingTriangle().
- If choice is 2, call naturalNumberPattern().
- If choice is 3, call repeatedNumberPattern().
- If choice is 4, call reverseNumberTriangle().
- If choice is 5, terminate the program.
- Otherwise, display "Invalid choice".

Step 6: Repeat the menu until the user selects Exit.

Step 7: Stop.

Algorithm for Each Pattern:

Pattern 1: Decreasing Right-Angled Triangle

1. Start the function.
2. Input number of rows n.
3. If $n \leq 0$, display an error message and return.
4. Run outer loop from $i = n$ down to 1.
5. Run inner loop from $j = 1$ to i .
6. Print * in the inner loop.
7. After the inner loop, move to the next line.
8. Stop the function.

Pattern 2: Decreasing Natural-Number Pattern

1. Start the function.
2. Input n.

3. If $n \leq 0$, display an error message and return.
4. Run outer loop from $i = n$ down to 1.
5. For every row, run inner loop from $j = 1$ to i .
6. Print the value of j .
7. Move to the next line after every row.
8. Stop the function.

Pattern 3: Decreasing Repeated-Number Pattern

1. Start the function.
2. Input n .
3. If $n \leq 0$, display an error message and return.
4. Run outer loop from $i = n$ down to 1.
5. Run inner loop from $j = 1$ to i .
6. Print i in the inner loop.
7. Move to the next line after every row.
8. Stop the function.

Pattern 4: Reverse-Number Triangle

1. Start the function.
2. Input n .
3. If $n \leq 0$, display an error message and return.
4. Run outer loop from $i = n$ down to 1.
5. Run inner loop from $j = i$ down to 1.
6. Print j .
7. Move to the next line after every row.
8. Stop the function.

C PROGRAM:

```
#include <stdio.h>
```

```
/* Function declarations */
```

```
void decreasingTriangle(void);
void naturalNumberPattern(void);
void repeatedNumberPattern(void);
void reverseNumberTriangle(void);

int main()
{
    int choice;

    do
    {
        printf("\n--- Decreasing Patterns Menu ---\n");
        printf("1. Decreasing right-angled triangle\n");
        printf("2. Decreasing natural-number pattern\n");
        printf("3. Decreasing repeated-number pattern\n");
        printf("4. Reverse-number triangle\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                decreasingTriangle();
                break;

            case 2:
```

```
        naturalNumberPattern();

        break;

    case 3:

        repeatedNumberPattern();

        break;

    case 4:

        reverseNumberTriangle();

        break;

    case 5:

        printf("Exiting program...\n");

        break;

    default:

        printf("Invalid choice. Please enter 1 to 5.\n");

    }

} while (choice != 5);

return 0;

}

/* Pattern 1: Decreasing right-angled triangle */
void decreasingTriangle(void)
{
```

```
int n, i, j;

printf("Enter the number of rows: ");
scanf("%d", &n);

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = n; i >= 1; i--)
{
    for (j = 1; j <= i; j++)
    {
        printf("* ");
    }

    printf("\n");
}
}
```

```
/* Pattern 2: Decreasing natural-number pattern */
void naturalNumberPattern(void)
{
    int n, i, j;
```

```

printf("Enter the number of rows: ");
scanf("%d", &n);

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = n; i >= 1; i--)
{
    for (j = 1; j <= i; j++)
    {
        printf("%d ", j);

    }

    printf("\n");
}
}

/* Pattern 3: Decreasing repeated-number pattern */
void repeatedNumberPattern(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

```

```
if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = n; i >= 1; i--)
{
    for (j = 1; j <= i; j++)
    {
        printf("%d ", i);

        printf("\n");
    }
}
```

/* Pattern 4: Reverse-number triangle */

```
void reverseNumberTriangle(void)
```

```
{
```

```
    int n, i, j;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &n);
```

```
    if (n <= 0)
```

```

{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = n; i >= 1; i--)
{
    for (j = i; j >= 1; j--)
    {
        printf("%d ", j);
    }

    printf("\n");
}
}

```

TEST CASE TABLE:

TEST CASE	MENU CHOICE	INPUT(n)	EXPECTED RESULT
1	1	4	Prints decreasing * triangle with 4 rows
2	2	4	Prints 1 2 3 4, then 1 2 3, 1 2, 1
3	3	4	Prints 4 4 4 4, then 3 3 3, 2 2, 1
4	4	4	Prints 4 3 2 1, then 3 2 1, 2 1, 1
5	1	1	Prints one *
6	2	0	Displays "Number of rows must be positive."
7	3	-2	Displays "Number of rows must be positive."
8	7	--	Displays "Invalid choice."
9	5	--	Exits the program

```
— Decreasing Patterns Menu —
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
```

Enter your choice: 1

Enter the number of rows: 4

```
* * * *
* * *
* *
*
```

```
— Decreasing Patterns Menu —
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
```

Enter your choice: 2

Enter the number of rows: 4

```
1 2 3 4
1 2 3
1 2
1
```

```
— Decreasing Patterns Menu —
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
```

Enter your choice: 3

Enter the number of rows: 4

```
4 4 4 4
3 3 3
2 2
1
```

```
— Decreasing Patterns Menu —
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
```

Enter your choice: █